

Guia Completo dos Comandos do Claude Code

Referência em português de **todos os comandos** do Claude Code: comandos de barra (/), comandos de terminal (`claude ...`), atalhos de teclado e modos especiais.

Baseado na documentação oficial (versão 2.1.x) — code.claude.com/docs — Atualizado em 23/06/2026.

Como usar: dentro do Claude Code digite / para ver os comandos disponíveis. No terminal (PowerShell/CMD) digite `claude ...`. Nem todo comando aparece para todos — depende do plano, sistema e ambiente.

Índice

1. Comandos de barra (/) — dentro do Claude Code

2. Comandos de terminal (CLI) — no PowerShell

3. Flags (opções) da linha de comando

4. Atalhos de teclado
5. Modos especiais e prefixos

6. Modo Vim (edição estilo Vim)

7. "O que isso significa pra mim" — guia rápido do dia a dia

1. Comandos de barra (/) — dentro do Claude Code

Digitados no início da mensagem. O texto após o comando vira "argumento". `<arg>` = obrigatório, `[arg]` = opcional.

Configuração e início de projeto

Comando	O que faz (em português)
<code>/init</code>	Cria um arquivo <code>CLAUDE.md</code> inicial documentando o seu projeto (guia para o Claude).
<code>/memory</code>	Edita os arquivos de memória <code>CLAUDE.md</code> ; liga/desliga a memória automática e mostra suas entradas.
<code>/config [chave=valor]</code>	Abre as configurações (tema, modelo, estilo de saída...). Pode setar direto, ex.: <code>/config theme=dark</code> . Apelido: <code>/settings</code> .
<code>/permissions</code>	Gerencia regras de permissão (permitir / perguntar / negar) das ferramentas. Apelido: <code>/allowed-tools</code> .
<code>/agents</code>	Cria e gerencia subagentes de IA (delegação de tarefas).
<code>/mcp</code>	Gerencia conexões com servidores MCP e autenticação OAuth.
<code>/hooks</code>	Vê e configura os <i>hooks</i> (ganchos) que disparam em eventos de ferramentas.
<code>/skills</code>	Lista as <i>skills</i> (habilidades) disponíveis; permite ocultar alguma.
<code>/plugin [subcomando]</code>	Gerencia plugins (<code>list</code> , <code>install</code> , <code>enable</code> , <code>disable</code> ...).
<code>/reload-plugins</code>	Recarrega plugins sem reiniciar (use <code>--force</code> se mudar ferramentas MCP).
<code>/reload-skills</code>	Re-escaneia skills/comandos adicionados durante a sessão.
<code>/statusline</code>	Configura a barra de status do Claude Code.
<code>/keybindings</code>	Abre o arquivo de atalhos de teclado para personalizar.
<code>/theme</code>	Muda o tema de cores (claro, escuro, automático, daltônico...).
<code>/terminal-setup</code>	Configura o atalho Shift+Enter (nova linha) no seu terminal (VS Code, Cursor etc.).
<code>/ide</code>	Gerencia a integração com a IDE e mostra o status.

Durante a tarefa

Comando	O que faz (em português)
---------	--------------------------

<code>/plan [descrição]</code>	Entra no modo de planejamento (planeja antes de executar). Ex.: <code>/plan corrigir o bug de login</code> .
<code>/model [modelo]</code>	Troca o modelo de IA (Opus, Sonnet, Haiku, Fable) e salva como padrão.
<code>/effort [nível]</code>	Ajusta o "esforço" de raciocínio: <code>low</code> , <code>medium</code> , <code>high</code> , <code>xhigh</code> , <code>max</code> , <code>ultracode</code> .
<code>/fast [on off]</code>	Liga/desliga o modo rápido (mesmo Opus, com saída mais veloz).
<code>/context [all]</code>	Mostra o uso do contexto (janela) num grid colorido, com dicas de otimização.
<code>/compact [instruções]</code>	Resume a conversa para liberar contexto. Pode focar o resumo em algo.
<code>/clear [nome]</code>	Começa uma conversa nova com contexto vazio (mantém a memória do projeto). Apelidos: <code>/reset</code> , <code>/new</code> .
<code>/btw <pergunta></code>	Pergunta rápida "a propósito" sem poluir o histórico da conversa.
<code>/goal [condição]</code>	Define uma meta: o Claude continua trabalhando entre turnos até cumprir.
<code>/recap</code>	Gera um resumo de uma linha do que aconteceu na sessão.
<code>/copy [N]</code>	Copia a última resposta do Claude (ou a N-ésima) para a área de transferência.
<code>/export [arquivo]</code>	Exporta a conversa como texto (arquivo ou área de transferência).
<code>/diff</code>	Abre um visualizador interativo das mudanças não commitadas e por turno.
<code>/focus</code>	Mostra só o essencial (seu último prompt + resumo das ações + resposta final).
<code>/insights</code>	Gera um relatório analisando suas sessões (padrões, pontos de atrito).

Nota: `/plan` × `Shift+Tab` × `/ultraplan` — as três formas de planejar. Todas levam ao **modo de planejamento** (o Claude propõe um plano e só executa após sua aprovação). A diferença é só *como* você entra nele:

- `/plan [descrição]` — comando que entra no modo já com a tarefa. Ex.: `/plan corrigir o bug de login`.
- `Shift+Tab` — atalho de teclado que alterna para o modo de planejamento (sem digitar).
- `/ultraplan <prompt>` — o "primo grande": rascunha o plano numa sessão **na nuvem**, você revisa no navegador e manda executar. Para tarefas maiores.

Trabalho em paralelo e em segundo plano

Comando	O que faz (em português)
<code>/background [prompt]</code>	Destaca a sessão para rodar como agente em segundo plano , liberando o terminal. Apelido: <code>/bg</code> .
<code>/tasks</code>	Vê e gerencia tudo que está rodando em segundo plano. Também como <code>/bashes</code> .
<code>/fork <diretriz></code>	Cria um subagente que herda a conversa e trabalha numa tarefa enquanto você continua.
<code>/branch [nome]</code>	Ramifica a conversa neste ponto para tentar outra direção sem perder a original.
<code>/batch <instrução></code>	(Skill) Decompõe uma mudança grande em 5–30 unidades e roda cada uma em paralelo (worktrees).
<code>/loop [intervalo] [prompt]</code>	(Skill) Roda um prompt repetidamente. Ex.: <code>/loop 5m verifique se o deploy terminou</code> . Apelido: <code>/proactive</code> .
<code>/workflows</code>	Abre a visão de progresso dos <i>workflows</i> (orquestração de vários subagentes).
<code>/stop</code>	Para a sessão em segundo plano à qual você está conectado.

Antes de "entregar" (revisão de código)

Comando	O que faz (em português)
<code>/code-review [nível] [--fix] [--comment] [alvo]</code>	(Skill) Revisa o <i>diff</i> atual buscando bugs e melhorias. <code>--fix</code> aplica correções; <code>ultra</code> roda revisão profunda na nuvem.
<code>/review [PR]</code>	Revisa um Pull Request do GitHub (mesma engine do <code>/code-review</code> , só leitura).

<code>/simplify [alvo]</code>	(Skill) Revisa o código só para limpeza/simplificação e aplica os ajustes (não caça bugs).
<code>/security-review</code>	Analisa as mudanças pendentes em busca de vulnerabilidades de segurança.
<code>/ultrareview [PR]</code>	Revisão profunda multiagente na nuvem. Prefira <code>/code-review ultra</code> (este é apelido).
<code>/ultraplan <prompt></code>	Rascunha um plano numa sessão na nuvem, revisa no navegador e executa.
<code>/verify</code>	(Skill) Confirma que a mudança funciona rodando o app de verdade (não só testes).
<code>/run</code>	(Skill) Inicia e dirige o app do projeto para ver a mudança funcionando.
<code>/run-skill-generator</code>	(Skill) Ensina o <code>/run</code> e o <code>/verify</code> a iniciar o app do seu projeto.
<code>/autofix-pr [prompt]</code>	Cria sessão na nuvem que vigia o PR da branch e empurra correções quando o CI falha.
<code>/install-github-app</code>	Configura o app do Claude (GitHub Actions) no repositório.

Entre sessões / continuidade

Comando	O que faz (em português)
<code>/resume [sessão]</code>	Retoma uma conversa por ID/nome, ou abre o seletor de sessões. Apelido: <code>/continue</code> .
<code>/rename [nome]</code>	Renomeia a sessão atual (mostra o nome na barra).
<code>/rewind</code>	Volta a conversa e/ou o código a um ponto anterior (checkpoints). Apelidos: <code>/checkpoint</code> , <code>/undo</code> .
<code>/cd <caminho></code>	Move a sessão para um novo diretório de trabalho (preserva o cache).
<code>/add-dir <caminho></code>	Adiciona um diretório de trabalho para acesso a arquivos na sessão atual.
<code>/teleport</code>	Puxa uma sessão do "Claude Code na web" para este terminal. Apelido: <code>/tp</code> .
<code>/remote-control</code>	Permite controlar esta sessão local a partir do claude.ai. Apelido: <code>/rc</code> .
<code>/desktop</code>	Continua a sessão no app Claude Code Desktop. Apelido: <code>/app</code> .
<code>/remote-env</code>	Escolhe o ambiente padrão para agentes na nuvem.

Conta, ajuda e diagnóstico

Comando	O que faz (em português)
<code>/help</code>	Mostra a ajuda e a lista de comandos disponíveis.
<code>/login</code>	Entra na sua conta Anthropic.
<code>/logout</code>	Sai da sua conta Anthropic.
<code>/status</code>	Abre o painel de status (versão, modelo, conta, conectividade).
<code>/usage</code>	Mostra custo da sessão, limites do plano e estatísticas. Apelidos: <code>/cost</code> , <code>/stats</code> .
<code>/usage-credits</code>	Configura créditos de uso para continuar quando bater o limite.
<code>/doctor</code>	Diagnostica a saúde da instalação e das configurações (tecle <code>f</code> para o Claude corrigir).
<code>/debug [descrição]</code>	(Skill) Liga o log de depuração e investiga problemas da sessão.
<code>/feedback [relato]</code>	Envia feedback / reporta bug com o contexto da sessão. Apelidos: <code>/bug</code> , <code>/share</code> .
<code>/release-notes</code>	Vê o changelog (novidades) por versão.
<code>/privacy-settings</code>	Vê e ajusta configurações de privacidade (planos Pro/Max).
<code>/upgrade</code>	Abre a página para mudar para um plano superior.

<code>/heapdump</code>	Salva um snapshot de memória no <code>~/Desktop</code> para diagnosticar uso alto de memória.
------------------------	---

Skills e workflows agendados

Comando	O que faz (em português)
<code>/deep-research <pergunta></code>	(Workflow) Faz buscas na web, cruza fontes e gera um relatório com citações.
<code>/claude-api [migrate ...]</code>	(Skill) Carrega referência da Claude API para a linguagem do seu projeto.
<code>/fewer-permission-prompts</code>	(Skill) Cria uma allowlist no <code>settings.json</code> para reduzir pedidos de permissão.
<code>/schedule [descrição]</code>	Cria/edita/lista <i>routines</i> (tarefas) que rodam na nuvem em horário agendado. Apelido: <code>/routines</code> .
<code>/team-onboarding</code>	Gera um guia de integração para a equipe a partir do seu histórico de uso.
<code>/web-setup</code>	Conecta sua conta GitHub ao "Claude Code na web" usando o <code>gh</code> local.

Extras / interface

Comando	O que faz (em português)
<code>/tui [default fullscreen]</code>	Define o renderizador da interface (modo tela cheia sem flicker).
<code>/color [cor]</code>	Muda a cor da barra de prompt da sessão.
<code>/voice [hold tap off]</code>	Liga/desliga a ditação por voz (requer conta claude.ai).
<code>/scroll-speed</code>	Ajusta a velocidade de rolagem do mouse (modo tela cheia).
<code>/sandbox</code>	Liga/desliga o modo <i>sandbox</i> (isolamento).
<code>/advisor [modelo off]</code>	Liga/desliga a ferramenta "conselheira" (consulta um 2º modelo em momentos-chave).
<code>/chrome</code>	Configura a integração "Claude no Chrome".
<code>/powerup</code>	Lições interativas rápidas para descobrir recursos do Claude Code.
<code>/radio</code>	Abre a rádio lo-fi "Claude FM" no navegador.
<code>/mobile</code>	Mostra QR code para baixar o app móvel. Apelidos: <code>/ios</code> , <code>/android</code> .
<code>/passes</code>	Compartilha uma semana grátis de Claude Code com amigos (se elegível).
<code>/stickers</code>	Pede adesivos do Claude Code.
<code>/exit</code>	Sai do Claude Code. Apelido: <code>/quit</code> .

Observações: servidores MCP podem expor *prompts* que aparecem como comandos no formato `/mcp__<servidor>__<prompt>`. O `/vim` foi **removido** na v2.1.92 (use `/config` → Editor mode). O `/pr-comments` foi removido na v2.1.91 (peça direto ao Claude para ver os comentários do PR).

2. Comandos de terminal (CLI) — no PowerShell

Comando	O que faz (em português)
<code>claude</code>	Inicia uma sessão interativa.
<code>claude "pergunta"</code>	Inicia a sessão já com um prompt inicial.
<code>claude -p "pergunta"</code>	Modo "print": responde e sai (ideal para scripts).
<code>cat arquivo claude -p " ... "</code>	Processa conteúdo vindo de um <i>pipe</i> .
<code>claude -c</code>	Continua a conversa mais recente do diretório atual.

<code>claude -c -p " ... "</code>	Continua a conversa via modo print.
<code>claude -r "sessão" " ... "</code>	Retoma uma sessão por ID ou nome.
<code>claude update</code>	Atualiza para a última versão.
<code>claude install [versão]</code>	Instala/reinstala o binário nativo (ex.: <code>stable</code> , <code>latest</code> ou <code>2.1.118</code>).
<code>claude auth login</code>	Entra na conta Anthropic (<code>--console</code> para faturar via API; <code>--sso</code> para SSO).
<code>claude auth logout</code>	Sai da conta Anthropic.
<code>claude auth status</code>	Mostra o status de autenticação (JSON; <code>--text</code> para legível).
<code>claude agents</code>	Abre a "agent view" para monitorar/disparar sessões em segundo plano.
<code>claude attach <id></code>	Conecta-se a uma sessão em segundo plano neste terminal.
<code>claude logs <id></code>	Mostra a saída recente de uma sessão em segundo plano.
<code>claude respawn <id></code>	Reinicia uma sessão em segundo plano mantendo a conversa.
<code>claude stop <id></code>	Para uma sessão em segundo plano (também <code>claude kill</code>).
<code>claude rm <id></code>	Remove uma sessão em segundo plano da lista (transcrição fica salva).
<code>claude mcp</code>	Configura servidores MCP.
<code>claude mcp login <nome></code>	Faz o fluxo OAuth de um servidor MCP pela linha de comando.
<code>claude mcp logout <nome></code>	Limpa as credenciais OAuth de um servidor MCP.
<code>claude config</code>	Gerencia configurações pela linha de comando.
<code>claude plugin</code>	Gerencia plugins (apelido: <code>claude plugins</code>).
<code>claude project purge [caminho]</code>	Apaga o estado local do projeto (transcrições, logs, histórico...).
<code>claude remote-control</code>	Inicia um servidor de Controle Remoto (controlar via claude.ai/app).
<code>claude setup-token</code>	Gera um token OAuth de longa duração para CI e scripts.
<code>claude auto-mode defaults</code>	Imprime as regras do classificador do "auto mode" (JSON).
<code>claude daemon status</code>	Mostra o estado do supervisor de sessões em segundo plano.
<code>claude daemon stop --any</code>	Para o supervisor de sessões em segundo plano.
<code>claude ultrareview [alvo]</code>	Roda a ultrarrevisão de forma não-interativa (imprime os achados).
<code>claude --version (-v)</code>	Mostra o número da versão.
<code>claude --help (-h)</code>	Mostra a ajuda.

Se você digitar errado, o Claude Code sugere o mais próximo. Ex.: `claude udpate` → "Did you mean claude update?".

3. Flags (opções) da linha de comando

As mais úteis. (`claude --help` não lista todas — a ausência ali não significa indisponível.)

Modelo e raciocínio

Flag	O que faz
<code>--model <modelo></code>	Define o modelo da sessão (<code>sonnet</code> , <code>opus</code> , <code>haiku</code> , <code>fable</code> ou ID completo).
<code>--fallback-model <lista></code>	Modelo(s) reserva quando o principal está sobrecarregado/indisponível.
<code>--effort <nível></code>	Nível de esforço: <code>low</code> , <code>medium</code> , <code>high</code> , <code>xhigh</code> , <code>max</code> .

<code>--advisor <modelo></code>	Liga a ferramenta conselheira (<code>opus</code> , <code>sonnet</code> , <code>fable</code> ...).
---------------------------------------	--

Modo "print" / automação (scripts)

Flag	O que faz
<code>--print</code> , <code>-p</code>	Imprime a resposta sem modo interativo.
<code>--output-format <fmt></code>	Formato de saída: <code>text</code> , <code>json</code> , <code>stream-json</code> .
<code>--input-format <fmt></code>	Formato de entrada: <code>text</code> , <code>stream-json</code> .
<code>--json-schema '<schema>'</code>	Saída validada conforme um JSON Schema (modo print).
<code>--max-turns <n></code>	Limita o número de turnos (modo print).
<code>--max-budget-usd <valor></code>	Teto de gasto em dólares antes de parar (modo print).
<code>--bare</code>	Modo mínimo: pula descoberta de hooks/skills/plugins/MCP/CLAUDE.md (mais rápido).
<code>--verbose</code>	Log detalhado, turno a turno.

Permissões e ferramentas

Flag	O que faz
<code>--permission-mode <modo></code>	Inicia em um modo: <code>default</code> , <code>acceptEdits</code> , <code>plan</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code> .
<code>--dangerously-skip-permissions</code>	⚠️ Pula todos os pedidos de permissão. Use com muito cuidado.
<code>--allow-dangerously-skip-permissions</code>	Adiciona o <code>bypassPermissions</code> ao ciclo do Shift+Tab sem iniciar nele.
<code>--allowedTools " ... "</code>	Ferramentas que executam sem pedir permissão.
<code>--disallowedTools " ... "</code>	Regras de negação de ferramentas.
<code>--tools "Bash,Edit,Read"</code>	Restringe quais ferramentas embutidas o Claude pode usar.

Diretórios, sessão e contexto

Flag	O que faz
<code>--add-dir <caminho></code>	Adiciona diretórios de trabalho.
<code>--resume <id></code> , <code>-r</code>	Retoma uma sessão específica (ou abre o seletor).
<code>--continue</code> , <code>-c</code>	Carrega a conversa mais recente do diretório.
<code>--fork-session</code>	Ao retomar, cria um novo ID em vez de reutilizar o original.
<code>--session-id <uuid></code>	Usa um ID de sessão específico.
<code>--name <nome></code> , <code>-n</code>	Define um nome de exibição para a sessão.
<code>--worktree <nome></code> , <code>-w</code>	Inicia em um <i>git worktree</i> isolado.
<code>--ide</code>	Conecta automaticamente à IDE no início.
<code>--chrome</code> / <code>--no-chrome</code>	Liga/desliga a integração com o Chrome.

MCP, plugins e prompt do sistema

Flag	O que faz
<code>--mcp-config <arquivo></code>	Carrega servidores MCP de arquivos/strings JSON.

<code>--strict-mcp-config</code>	Usa só os servidores MCP do <code>--mcp-config</code> .
<code>--plugin-dir <caminho></code>	Carrega um plugin de uma pasta/ <code>.zip</code> só nesta sessão.
<code>--settings <arquivo></code>	Caminho (ou JSON) de configurações que sobrescrevem o <code>settings.json</code> .
<code>--system-prompt " ... "</code>	Substitui todo o prompt do sistema.
<code>--append-system-prompt " ... "</code>	Acrescenta texto ao final do prompt do sistema padrão.
<code>--safe-mode</code>	Inicia com todas as customizações desativadas (para diagnosticar).
<code>--debug "api,mcp"</code>	Liga o modo de depuração com filtro de categorias.

4. Atalhos de teclado

Controles gerais

Atalho	O que faz
<code>Esc</code>	Interrompe o Claude no meio do turno (mantém o trabalho feito).
<code>Esc Esc</code>	Com texto digitado: limpa e salva no histórico. Vazio: abre o menu de <i>rewind</i> (checkpoints).
<code>Ctrl+C</code>	Interrompe a operação; ou limpa a entrada; de novo (vazio) sai do Claude Code.
<code>Ctrl+D</code>	Sai da sessão do Claude Code (sinal EOF).
<code>Ctrl+L</code>	Redesenha a tela (recupera display embaralhado).
<code>Ctrl+O</code>	Liga/desliga o visualizador de transcrição (detalhes de ferramentas).
<code>Ctrl+R</code>	Busca reversa no histórico de comandos.
<code>Ctrl+B</code>	Joga a tarefa/comando atual para segundo plano (tmux: duas vezes).
<code>Ctrl+T</code>	Liga/desliga a lista de tarefas.
<code>Ctrl+G</code> ou <code>Ctrl+X Ctrl+E</code>	Abre o prompt no seu editor de texto padrão.
<code>Ctrl+V</code> / <code>Alt+V</code> (Win/WSL)	Cola imagem da área de transferência (insere chip <code>[Image #N]</code>).
<code>Ctrl+X Ctrl+K</code>	Para todos os subagentes em segundo plano (duas vezes em 3s para confirmar).
<code>Shift+Tab</code> (ou <code>Alt+M</code>)	Alterna os modos de permissão (<code>default</code> , <code>acceptEdits</code> , <code>plan</code> , <code>auto</code> ...).
<code>Alt+P</code> (Win/Linux)	Troca o modelo sem limpar o prompt.
<code>Alt+T</code> (Win/Linux)	Liga/desliga o <i>extended thinking</i> (raciocínio estendido).
<code>Alt+O</code> (Win/Linux)	Liga/desliga o modo rápido .
<code>↑</code> / <code>↓</code> (ou <code>Ctrl+P</code> / <code>Ctrl+N</code>)	Move o cursor; nas bordas, navega o histórico de comandos.
<code>←</code> / <code>→</code>	Navega entre abas de diálogos/menus.

Edição de texto

Atalho	O que faz
<code>Ctrl+A</code>	Vai para o início da linha.
<code>Ctrl+E</code>	Vai para o fim da linha.
<code>Ctrl+K</code>	Apaga até o fim da linha (guarda para colar).
<code>Ctrl+U</code>	Apaga do cursor até o início da linha (guarda para colar).

<code>Ctrl+W</code>	Apaga a palavra anterior (no Windows, <code>Ctrl+Backspace</code> também).
<code>Ctrl+Y</code>	Cola o texto apagado com <code>Ctrl+K/U/W</code> .
<code>Alt+B</code> / <code>Alt+F</code>	Movimenta o cursor uma palavra para trás / para frente.

Nova linha (entrada multilinha)

Atalho	Contexto
<code>\ + Enter</code>	Funciona em qualquer terminal.
<code>Ctrl+J</code>	Funciona em qualquer terminal, sem configurar.
<code>Shift+Enter</code>	Nativo em alguns terminais; nos demais (VS Code, Cursor...) rode <code>/terminal-setup</code> .

5. Modos especiais e prefixos

Prefixo / tecla	O que faz
<code>/</code> (no início)	Executa um comando ou skill.
<code>!</code> (no início)	Modo shell: roda um comando direto, adiciona a saída ao contexto e o Claude responde a ela. Ex.: <code>! npm test</code> , <code>! git status</code> .
<code>@</code>	Menção de arquivo: dispara o autocompletar de caminhos para referenciar no prompt.
Colar imagem	<code>Ctrl+V</code> / <code>Alt+V</code> insere a imagem para você referenciar no texto.

O modo shell (`!`) sai com `Esc`, `Backspace` ou `Ctrl+U` no prompt vazio, e aceita autocompletar com `Tab` a partir de comandos `!` anteriores do projeto.

6. Modo Vim (edição estilo Vim)

Ative em `/config` → **Editor mode**. Principais comandos no modo NORMAL:

- Trocar de modo:** `Esc` (NORMAL), `i` / `I` (inserir antes/início), `a` / `A` (inserir depois/fim), `o` / `O` (abrir linha abaixo/acima), `v` / `V` (seleção visual).
- Navegar:** `h j k l` (←↓↑→), `w` (próxima palavra), `e` (fim da palavra), `b` (palavra anterior), `0` (início), `$` (fim), `^` (1º não-branco), `gg` / `G` (início/fim), `{c}` / `C{c}` (pular para caractere).
- Editar:** `x` (apaga caractere), `dd` (apaga linha), `D` (apaga até o fim), `dw/cw/yw` (apagar/mudar/copiar palavra), `cc` (muda linha), `yy` / `Y` (copia linha), `p` / `P` (cola depois/antes), `u` (desfaz), `.` (repete), `>>` / `<<` (indenta/desindenta).
- Objetos de texto** (com `d`, `c`, `y`): `iw` / `aw` (palavra), `is` / `as` (aspas), `ic` / `ac` (parênteses), `if` / `af` (chaves), etc.

7. "O que isso significa pra mim" — guia rápido do dia a dia

Tradução prática dos comandos que mais valem a pena no uso comum:

- Conversa ficou enorme e lenta?** `/compact` resume e libera espaço **mantendo** a conversa; `/clear` começa **do zero** (mantém só a memória do projeto) — use ao trocar de assunto; `/context` mostra para onde está indo a memória.
- Quer ver quanto está gastando?** `/usage` (ou `/cost`) mostra custo e limites do plano.
- Fechou o Claude e quer voltar de onde parou?** `claude -c` (no terminal) ou `/resume` (dentro). Para nomear sessões: `claude -n "meu-trabalho"`.
- Errou e quer desfazer mudanças no código/conversa?** `/rewind` volta a um ponto anterior (checkpoint). É a sua "rede de segurança".

- **Quer planejar antes de o Claude mexer em tudo?** `/plan` (ou `Shift+Tab` até o modo *plan*). Ele propõe e só executa após sua aprovação.
- **Algo parou de funcionar?** `/doctor` diagnostica a instalação; `/status` mostra versão, modelo e conexão.
- **Rodar um comando do sistema rápido sem sair da conversa?** Use `!` na frente. Ex.: `! ipconfig`.
- **Apontar um arquivo específico para o Claude olhar?** Digite `@` e comece a escrever o caminho.
- **Trocar o "capricho" da resposta:** `/model` (troca o modelo) e `/effort` (quão fundo ele pensa). `/fast` deixa o Opus mais ágil.
- **Acabar a tarefa:** `/exit` (ou `Ctrl+D`) para sair.

Gerado pelo Claude Code (engenheiro de informática pessoal do Edney) a partir da documentação oficial em code.claude.com/docs. Para a referência sempre atualizada, use `/help` dentro do Claude Code ou `claude --help` no terminal.